ECE 540: Now in 3D!

Ian Wyse, Paul Globisch, Nelson Rodriguez-Ortiz, Hunter Drake
Department of Electrical and Computer Engineering, Portland State University.
ECE 540, March 16th, 2026

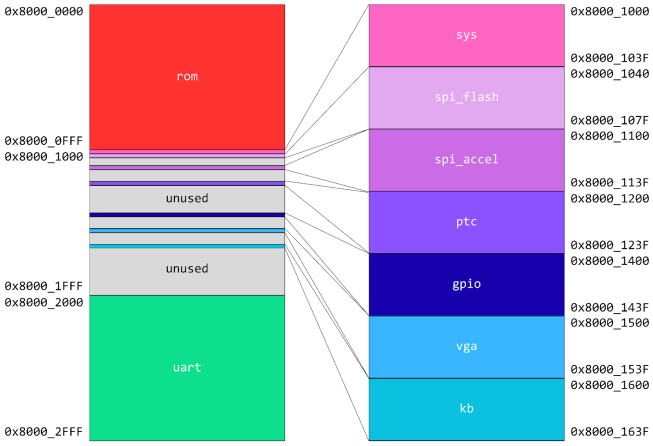
I. Introduction

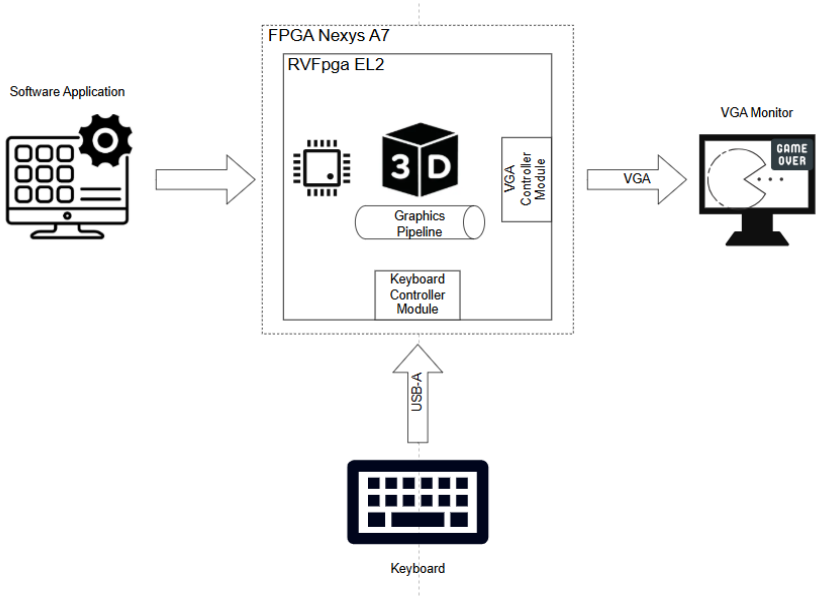
The process of taking 3D models and transforming them into a 2D space to be displayed on a screen (rendering) can be a very complex process. Modern graphics cards do this directly with a SIMT design philosophy that splits this massive problem into many smaller bite sized pieces for many arrays of small processors. This project demonstrates a more direct approach with purpose-built hardware for specific rendering functions.

II. Hardware

A. SoC Architecture

The System on Chip (SoC) was designed and implemented on the Nexys A7 T100 development board. The design consists primarily of a VeeRwolf EL2 core as the CPU and is connected to several peripherals. The connections to these peripherals are facilitated by AXI-to-Wishbone Bus Bridge. The AXI side of the bridge talks to the CPU, and the Wishbone side of the bridge talks with the peripherals. Though other peripherals exist on the SoC, the primary Wishbone devices that are used are the keyboard and GPU.





B. SoC Memory Map

The peripherals on this SoC can be accessed in between addresses 0x8000_0000 and 0x8000_2FFF. Below is a diagram of the peripheral memory map. Many of the peripherals on this map come with the VeeRwolf EL2 core by default, but this design comes with 2 additional devices. The device labeled “vga” is the GPU that interfaces with the

VGA monitor, and “kb” is the device that communicates with the plugged in keyboard for game control.

C. GPU Register Map

Memory mapped IO registers are provided to allow the CPU to communicate with the VGA GPU. A table of the available registers can be found below.

VGA GPU Register Map (Base Address = 0x8000_1500)	
Register Name	**Offset**
BRAM Buffer Index Register	0x00
Column Drawing Register	0x04
Frame Calculation Done Register	0x08
Status Register	0x0C
Control Register	0x10
Command Register	0x14

The BRAM Buffer Index Register is responsible for reporting which of the two BRAM buffers is currently displaying pixel data to the monitor. It contains a single bit that is either a ‘1’ or a ‘0’ as the index. This register is primarily used for handling BRAM buffer switches when frames are completed, but can be used for debugging purposes.

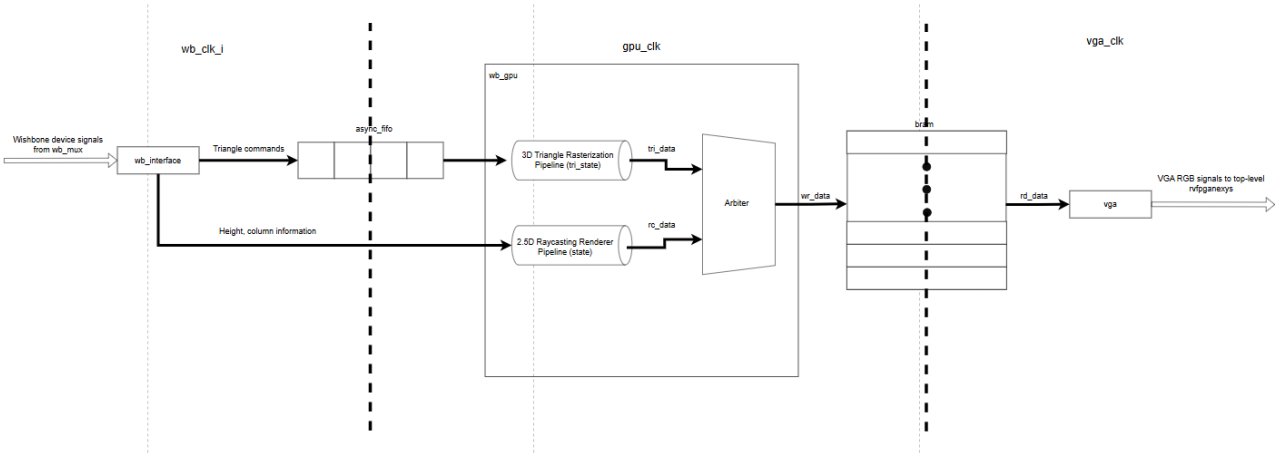
The Column Drawing Register is used to send wall column data to the GPU. The column index (9 bit), wall index (8 bit), and wall height (10 bit), are sent as a single 32-bit package. The column index is used to determine which pixel column to write to, the wall index is used to determine which address in the texture array to read, and the wall height is used to determine how many pixels should be wall pixels instead of ceiling or floor pixels.

The Frame Calculation Done Register (FCD) is used to tell the GPU that we have completed a frame and that it should be displayed. The GPU then switches which BRAM Buffer gets written to by the CPU and read from by the VGA monitor. Splitting frames into separate buffers prevents screen tearing, allowing for a better visual experience.

The Status Register is used to determine what is happening inside of the GPU. From it, the software can tell if the GPU Command Register FIFO is full, empty, overflown, or if the GPU is busy.

The Control Register is used to control the behavior of the GPU. One of the bits in the register controls if triangles can be written over columns, another one controls whether the GPU is allowed to parse Command Register data. As far as this project is concerned, all that needs to happen to/with this register is for both of the bits to be written to ‘1’ to enable both primitives to be written to the BRAM and for triangles to be drawn over columns at the beginning of the program.

The Command Register is used to send triangle data to the GPU so that they can be rasterized. The data sent to this register includes triangle point data, color, and height. Height in this sense is not referring to the triangle’s geometry, but rather refers to the draw order. The way that all this data can fit into this register is by chopping the data into 4-byte instructions that are called “commands”. Each command has a field where the operation code (op-code) goes which tells the GPU what type of data this is and thus how to process it. In order to start the triangle rasterization process after all the



triangle's points have been written to the GPU, the triangles color and height are written. This command registers all the triangle's data in the GPU and starts the rasterization process.

D. GPU Architecture

The GPU architecture operates in three clock domains:

- wb_clk_i – 12.5MHz
- gpu_clk – 100MHz
- vga_clk – 25MHz

The architecture consists of a subsystem with the following components:

- Wishbone bus register interface – Handles register reads and writes requests from the Veerwolf core through the Wishbone interface.
- Asynchronous FIFO – Handles clock domain crossing from the wb_clk_i to the gpu_clk and queues the incoming triangle commands sent from the application.
- 2.5D raycasting renderer pipeline – FSM that draws the 2.5D scene walls and implements texture mapping for colors.
- 3D triangle rasterization pipeline – FSM that dequeues commands from async FIFO and performs triangle rasterization to draw the 3D object.
- Block RAM (BRAM) – Handles the clock domain crossing from gpu_clk to vga_clk and stores the pixel data in a 320x240 frame buffer. It is double buffered to prevent screen tearing. A simple arbitration is done to allow writing the 2.5D scene pixels and the 3D object pixels.
- VGA controller– Similar implementation as ECE540 Project 2, but additionally it performs scaling on frame buffer from 320x240 to 640x480 for VGA display. It reads one pixel from the frame BRAM every vga_clk clock cycle.

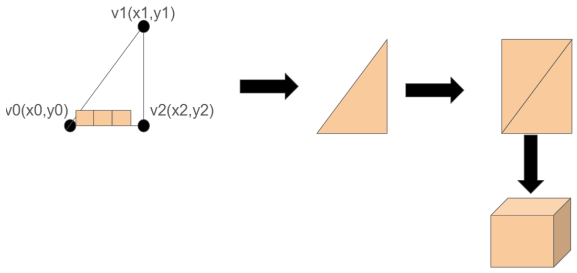
E. Triangle Rasterization for 3D Rendering

3D objects are rendered using triangle rasterization. To accomplish this, the software application will write 4 commands into the Command Register per triangle:

- Command 1 – v0 coordinates
- Command 2 – v1 coordinates
- Command 3 – v2 coordinates
- Command 4 – color and height

The object selected to be drawn was a 3D cube due to its simplicity. A cube has 6 faces and each face is rasterized with 2 triangles, therefore, the total number of

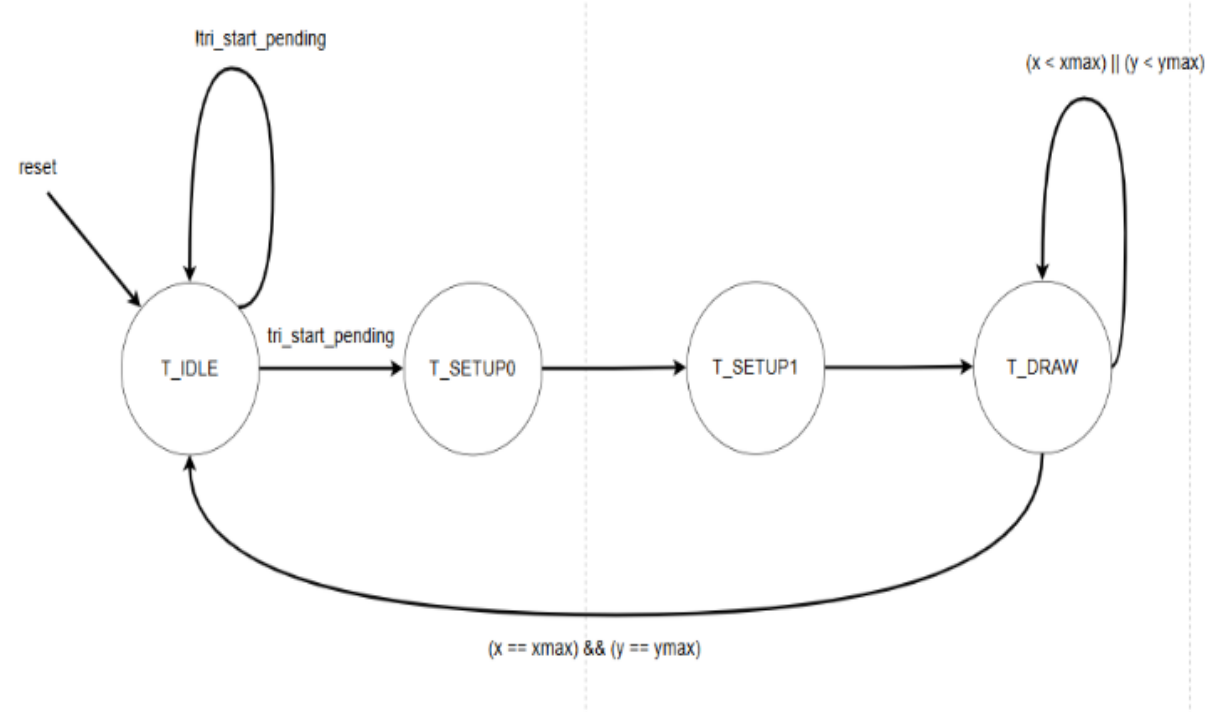
triangle commands needed to render one cube is 48 commands. The async FIFO was sized to the next power of 2 number after 48 to avoid overflowing the FIFO often and possibly harming performance.



The sides of a triangle are called edges. Triangles are rasterized using edge functions in each of its sides. The algorithm follows the equations:

$$E01(x,y)=(y-y0)(x1-x0)-(x-x0)(y1-y0)$$
$$E12(x,y)=(y-y1)(x2-x1)-(x-x1)(y2-y1)$$
$$E20(x,y)=(y-y2)(x0-x2)-(x-x2)(y0-y2)$$

For a triangle, a point is inside if all three edge functions have the same sign. Typically, the edge functions calculations are done for every pixel which involves intensive computation that is not suitable in synthesizable hardware to meet timing requirements. In real GPU graphics pipelines, this computation is divided into tasks that will be computed by different threads in parallel using the SIMT approach. However, the graphics pipeline in this project was redesigned to handle the edge functions computations incrementally with each step done in one gpu_clk clock cycle. In a real-life graphics application this might degrade the user's experience and performance, but for the proof of concept being done for this project it is an acceptable solution. Specifically, edge functions deltas (example: dx01 = x1-x0, dy01 = y1-y0) are pre-computed, multiplies are done in one pipe stage and increments to the next edge function across x and y are done in another pipe stage. An FSM was implemented to track each of the stages of this pipeline.



F. 2.5D Raycasting

The hardware portion of the raycasting and wall rendering begins with the arrival of Wishbone data. Each data packet contains the column address, wall index, and wall height for that column. These values are 9, 8, and 10 bits respectively.

The arrival and synchronization of a data packet triggers the column writing FSM to transition from IDLE state to DRAWING state. The drawing state persists for 240 cycles (one for each pixel in the column). Three pipelined tasks are performed in each iteration of the DRAWING state. They are:

Calculations: In this step, the frame buffer and texture addresses are calculated. In retrospect, this logic could have been done in the same clock cycle as the memory accesses for each of the BRAM tables, but in an abundance of caution to avoid timing failures, this logic was spun off into its own clock cycle.

Texture Array Access: The texture array is accessed using the computed address to determine what color the wall should be. This is performed regardless of whether the pixel is a wall, ceiling, or floor pixel, though if it is a ceiling or floor pixel the result is not used.

Note: The wall height can range from 0 to 1023, which is significantly larger than the screen, which is only 240 pixels tall. This is done so that when the player is very close to a wall, textures still function properly. If the height caps at 255, textures will become static at any distance closer than would result in a height of 255, which causes them to stretch horizontally but not vertically, which looks very bad. By allowing larger heights, the textures can stretch in both directions, allowing the approach to appear natural.

Frame Buffer Write: If the pixel is a wall pixel, the value read from the texture array is written out to the frame buffer. Otherwise either the static floor or ceiling color is written.

In parallel to this functionality, the height of the current column is written to the z-buffer at the column index address. The functionality of the z-buffer is described in the next section.

G. Object Ordering Logic

When rendering objects at different distances, where nearer objects may occlude further ones, a standard practice is to render objects starting with the most distant, ending with the nearest. This has the natural effect of placing nearer objects in front of more distant ones. This method cannot be used with walls and cubes. Since walls are rendered first, with cubes rendered “on top” of them, a different method must be used in order to prevent cubes from being visible despite being physically behind a nearer wall. The methodology for doing so is the z-buffer, or height buffer.

As mentioned in section 2, whenever a column data packet arrives, the height value is written to a BRAM z-buffer, with address determined by the column address in the packet. When triangles are sent to the rasterization side of the GPU, they are sent with a height value, which is calculated in software in the same way as the column height, and represents the inverse of the distance from the camera to that triangle. When triangles are being written to the BRAM, the “height” of each pixel is compared to the z-buffer value for that pixel’s column. If the height is smaller than the read value, it means that portion of the triangle is behind a wall, and should not be

rendered. This allows for cubes to smoothly transition from visible to not-visible as they pass behind walls.

H. The VGA and Frame Buffer Modules

The frame buffer module is where the current and next frame data are stored. It contains two frames. At any given time, the VGA module is reading from one of them, and the GPU is writing to the other. When the GPU finishes writing to its frame, it sets a frame-done signal high. Whenever the VGA module finishes reading a frame, it checks to see if this frame-done signal is set. If so, it swaps frames, so that it is reading the newly written frame and the GPU is writing to the now unused frame. Each frame buffer contains 12-bit color data for every pixel in the 320x240 display.

The VGA module is similar to the one used in project 2, with the change that every clock cycle it outputs the appropriate pixel from the frame buffer, rather than a character pixel. Since BRAM access has a one cycle latency, signals from the dtg.sv module are pipelined/delayed two clock cycles to align the signal with the correct pixel from the BRAM. As previously mentioned, whenever a frame is completed, the module checks to see if the GPU has finished writing its frame, and if so swaps the read and write frames.

III. Software

A. Cube Rendering

Cubes are rendered as simple 3D meshes on top of the raycasted scene. Each cube is represented by a set of 8 vertices composing 6 quad faces. Each visible quad is split into 2 triangles before being set to the GPU.

Per-frame, the following functions are used in the engine to process the cubes. get_cube_camera_offsets computes each cube's position relative to the player. sort_cubes orders cubes by depth, so the more distant cubes can be drawn first. render_cube applies transformations for each cube and decomposes them into a series of triangle commands. update_cube applies motion and rotation for the next frame.

The rendering code uses fixed-point with 8 fractional bits for all mesh operations, as well as lighting calculations. The process of rendering a cube involves starting with a given vertex in local space, applying the rotation of the cube and position offsets, then subtracting the camera position to move into camera space, followed by projecting each vertex onto the 2D screen space.

Before a cube begins rendering, cubes fully behind the camera are ignored. This is followed by an approximate check for cubes outside the horizontal field of view. These early tests reduce the cost of rendering multiple cubes each frame, especially for larger scenes. During projection, points too close to the camera are also rejected. For a face to be drawn, all four of its face vertices must be in front of the near clip plane, at least one projected point must be valid, and the face must pass back-face culling. Determination of if a face is visible is done by calculating the normal of the face and comparing it against the face center. This means that for a cube, typically only 3 faces, or 6 triangles, are rasterized at once.

Due to memory constraints, no true software Z-buffer is used for rendering faces. A simple Painter's Algorithm is used to prevent faces from overlapping incorrectly; faces are just drawn back to front, so the final image looks accurate. A simple 1-D Z-buffer is implemented in

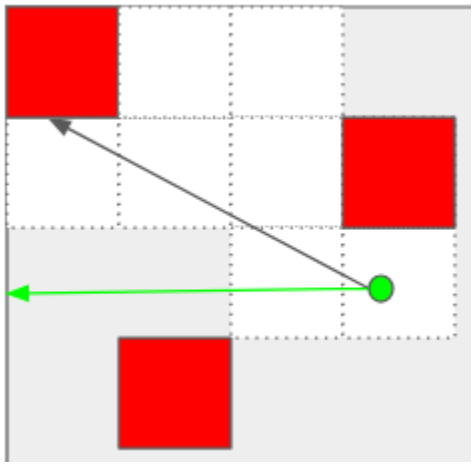
hardware to manage wall/triangle render ordering.

The lighting on 3D meshes is directional lighting from a single global source. Faces with a normal that aligns with the incident light source are rendered at their full color value. The more a face faces away, the more its color darkens, until it reaches the ambient brightness.

B. Raycasting

Raycasting is managed by the updateRaycasting function, which lives in the raycasting.c file. The core functionality of raycasting is relatively simple. For each pixel column, a line is cast forward from the player's current position. It travels forward through space until it hits a wall. The distance traveled determines the rendered height of the wall for that pixel column.

The original algorithm was based on code written by Nick Stones-Havas [1], but is heavily modified to function in an FPGA system, including the removal of all floating point calculations and other optimization efforts.



The behavior of the function to implement this is as follows. First, the player's world coordinates are converted to map coordinates, which are used for raycasting. Then the player direction is calculated, and the FOV is determined. The raycasting loop begins with the leftmost column in the FOV.

The first step in the raycasting loop is to determine the location and orientation of the nearest gridline along the ray's path out from the player. This must be done separately, as it is not guaranteed that the player is standing on a gridline.

From there, the loop is simple. All that must be done is to determine whether the cast ray collides with an x-gridline or a y-gridline first, and where on that gridline the collision occurs. The ability to jump from gridline to gridline makes this process manageable even with a 12.5MHz processor.

The loop ends when a wall is found. At this point, the location of the wall is known, so the distance from the wall to the player is calculated. There is some nuance here. If the exact distance to the player was used, the result would be a "fisheye" effect. Imagine looking straight at a flat wall. The distance to the camera is shorter in the center of the frame than on the edges, so the walls are shorter there, even though the wall is straight and they should appear to be the same height.

The solution is to calculate the distance between the wall and the camera plane, a "screen" that is normal to the player's view direction. This prevents the fisheye effect from occurring because straight walls (or similar) all end up with the same distance to the camera plane.

This distance is converted to a height, and the position on the wall where the ray struck is calculated for use in texture mapping. These values are packed, along with the ray index, and they are sent to the GPU for processing. The process then starts over with the next column's ray.

C. Entity Management

Entity management consists of determining the position and behavior of the player and cubes. Functions, constants, and data structures pertaining to this task are contained in the player.c and player.h files.

When the program first runs or when the game is restarted, the initPlayer function is called. This function iterates over all grid elements in the environment array. When it finds one containing the player start position (indicated by a -1 in the array), it sets the player's x and y coordinates to the center of that square, and sets it facing due east.

Every game tick the player position is updated based on the user's keypresses. This is managed via the updatePlayer function. This function checks global moving and turning flags to see if the player should move forward, backwards, or turn right or left. Additionally it uses the player's current direction to determine how much their x and y coordinates should change should they be moving forward or backwards.

If one or both of the movement flags are set, the movePlayer function is called. This function takes in the dx and dy values calculated in the updatePlayer function and updates the global player position coordinates. It also performs a check to see if the motion would cause the player to run into a wall, and if it does, it prevents that portion of the motion.

If the rotate player flags are set, the rotatePlayer function is called. This function very simply increments the player angle value by a set amount defined in the config file.

The cube functions follow a similar pattern. The init_entities function iterates over the grid array looking for friend and foe cubes (2 and 4 respectively). If one is found, a cube is added to the world_cube array at the center of the square in which it is found, set to be either a friend (2) or foe (4). Each cube has a structure associated with it, which contains fields defining its position, orientation (yaw, pitch, roll), how each of those values should change each game tick, and whether it is enabled (should be rendered).

The update_cubes function updates the cube position and orientation values based on that cube's structure's deltas for those values. This function is called every game tick. Like with the player, cubes have a collision detection function, but unlike the player function, this function causes cubes to automatically change direction when they collide with a wall

There are also several helper functions for cube entities in this file. The count_cubes function counts the number of active friend or foe cubes. The sort_cubes function sorts an array of cubes by distance, furthest first, so that they can be rendered in that order.

Finally, the get_cubes_camera_offset function iterates over all cubes in the world list, determines which ones are in the player FOV, calculates their position and height relative to the player for rendering purposes, then places them in a list of cubes to be rendered for the current frame.

IV. Video Game

The actual video game implemented on top of this rendering software is quite simple. There are two types of cubes, friendly cubes and foe cubes. The user's goal is to touch (eat) all of the friendly cubes without letting any of the foe cubes touch them. It is akin to a 3d version of Pac-Man.

The foe cubes move constantly, bouncing off walls, while the friendly cubes are static.

If a foe cube touches the player, the player loses, a death screen animation plays, and the game is reset. If all friend cubes have been eaten, a winning screen animation plays, and again the game is reset.

The game behavior is implemented in the main.c file. When the game is launched, the init_game function is called, which calls the init_entities and initPlayer functions, then sets the foe cubes in motion and the friend cubes rotating.

Every game tick, the current time is read from the veerwolf timer module. This value is used to determine how far the player should move or rotate each game tick, as the duration of a game tick is determined by the render time of the current frame.

Input values are read from the keyboard, and the player's position is updated accordingly. This game uses the standard WASD player control method, W for forward, S for back, A and D for turn left and right.

After the player position is updated, the cube positions are updated, which may result in one or more cubes being removed from the game if they have collided with the player. Therefore the number of friend and foe cubes are counted.

If the number of foe cubes is less than there were originally, it means that one has touched the player, so the lose game screen is displayed and the game is restarted. Similarly, if there are no friend cubes remaining, the win game screen is displayed and the game is restarted.

If the game has not ended, raycasting is updated, the current positions of cubes in the player FOV are determined, and rendering is performed. The frame done function is then called, which waits for the VGA module to finish writing its current frame before continuing onto the next loop iteration.

V. Results

One of our goals in this project was to maintain high performance, measured in frames per second, in all of our renderings. Our target was maintaining 60 FPS on the VGA monitor, which is the maximum possible. We found that software latency dominated hardware latency as a bottleneck in this endeavor.

Testing on the cube mesh rendering and rasterization found that as many as 20 simultaneous cubes could be rasterized in one frame cycle, but that attempting to perform the software mesh calculations on more than five simultaneous cubes required an extra cycle.

This result was unsurprising, given the low clock speed (12.5MHz) of our CPU compared to the 100MHz available to our GPU. Since there are several occasions in gameplay where more than 5 cubes are present in the user's FOV, framerate drops occur, but fortunately the drop from 60 FPS to 30 FPS still results in a relatively smooth gameplay experience.

One avenue for potential future work involves improving FPS when multiple cubes are present. There are several approaches to explore in this regard. Currently, cubes present in the player FOV, but sitting behind walls, still go through the rendering process, they are just not displayed on screen. A software check to avoid rendering these cubes all together would improve performance when multiple cubes are present, but not visible.

Another potential path is attempting to streamline the mesh rendering code. For

instance, this could involve using additional LUTs instead of expensive operations, or replacing division operations with cheaper constant shifts. While it is not clear exactly how much performance improvement can be achieved via this method, it would certainly be the most straightforward method to achieve consistent improvements.

Lastly, we could attempt to offload computation that is currently in software onto custom hardware modules. This would be the most time consuming method of improving performance, but, if successful, would almost certainly result in the largest improvement.

Bibliography:

[1] Stones-Havas, Nick. *raycaster-sdl*. GitHub, 2018, github.com/drdanick/raycaster-sdl.